

JavaScript Math Object & Mathematical Operations

JavaScript-এর built-in `Math` object mathematical calculation করার জন্য অনেক useful methods এবং constants দেয়।

যেমন:

```
Math.floor()  
Math.ceil()  
Math.round()  
Math.random()  
Math.max()  
Math.min()
```

এর পাশাপাশি `Math.abs()`, `Math.sqrt()`, `Math.pow()`, `Math.trunc()`, `Math.sign()`, `Math.PI` ইত্যাদিও জানা গুরুত্বপূর্ণ।

1. What is `Math`?

`Math` হলো JavaScript-এর built-in object।
এটা ব্যবহার করে mathematical operations করা যায়।

```
console.log(Math.PI);  
console.log(Math.sqrt(25));  
console.log(Math.round(10.6));
```

`Math` সাধারণত constructor হিসেবে ব্যবহার করা হয় না।

```
new Math(); // ❌
```

বরং directly:

```
Math.floor(10.5);
```

2. `Math.floor()`

`Math.floor()` decimal value-কে নিচের **integer**-এর দিকে নিয়ে যায়।

```
Math.floor(10.9);  
// 10  
  
Math.floor(10.1);  
// 10
```

Negative number-এর ক্ষেত্রে সাবধান

```
Math.floor(-10.1);  
// -11
```

কারণ floor মানে zero-এর দিকে নয়, **negative infinity**-এর দিকে যাওয়া।

```
10.9    → 10  
10.1    → 10  
  
-10.1   → -11  
-10.9   → -11
```

3. `Math.ceil()`

`Math.ceil()` decimal value-কে উপরের **integer**-এর দিকে নিয়ে যায়।

```
Math.ceil(10.1);  
// 11  
  
Math.ceil(10.9);  
// 11
```

Negative:

```
Math.ceil(-10.1);  
// -10
```

কারণ ceil মানে positive infinity-এর দিকে যাওয়া।

4. `Math.round()`

`Math.round()` সবচেয়ে কাছের integer-এ round করে।

```
Math.round(10.4);  
// 10  
  
Math.round(10.5);  
// 11  
  
Math.round(10.8);  
// 11
```

সাধারণভাবে:

.0 - .4 → নিচে
.5 - .9 → উপরে

5. `floor()` vs `ceil()` vs `round()`

Value	<code>floor()</code>	<code>ceil()</code>	<code>round()</code>
10.1	10	11	10
10.4	10	11	10
10.5	10	11	11
10.8	10	11	11
-10.1	-11	-10	-10
-10.8	-11	-10	-11

সহজভাবে

floor → নিচে
ceil → উপরে
round → কাছের integer

6. `Math.trunc()`

`Math.trunc()` decimal অংশ সরিয়ে দেয়।

```
Math.trunc(10.99);  
// 10  
  
Math.trunc(10.1);  
// 10
```

Negative number:

```
Math.trunc(-10.99);  
// -10
```

এখানে `floor()` থেকে পার্থক্য:

```
Math.floor(-10.99);  
// -11  
  
Math.trunc(-10.99);  
// -10
```

Mental model

```
floor → নিচের integer  
trunc → decimal অংশ কেটে ফেলে
```

7. `Math.abs()`

`Math.abs()` absolute value দেয়।

```
Math.abs(10);  
// 10  
  
Math.abs(-10);  
// 10  
  
Math.abs(-99.5);  
// 99.5
```

Distance calculation-এ এটা অনেক useful।

```
const difference = Math.abs(100 - 130);  
  
console.log(difference);  
// 30
```

8. **Math.sign()**

কোনো number positive, negative, বা zero কিনা জানতে:

```
Math.sign(10);  
// 1  
  
Math.sign(-10);  
// -1  
  
Math.sign(0);  
// 0
```

আর:

```
Math.sign(-0);  
// -0
```

9. **Math.sqrt()**

Square root বের করতে:

```
Math.sqrt(25);  
    // 5  
  
Math.sqrt(100);  
    // 10
```

আর:

```
Math.sqrt(2);  
    // approximately 1.414...
```

10. `Math.pow()`

Power calculation:

```
Math.pow(2, 3);  
    // 8
```

মানে:

```
23 = 8
```

তবে modern JavaScript-এ exponentiation operator ব্যবহার করাও যায়:

```
2 ** 3;  
    // 8
```

তাই:

```
Math.pow(2, 3);
```

এর modern alternative:

```
2 ** 3;
```

11. **Math.PI**

Circle-related calculation-এর জন্য:

```
console.log(Math.PI);
```

প্রায়:

```
3.141592653589793
```

12. Circle Area

Formula:

$$\text{Area} = \pi r^2$$

JavaScript:

```
const radius = 5;

const area = Math.PI * radius ** 2;

console.log(area);
```

13. **Math.max()**

একাধিক number-এর মধ্যে সবচেয়ে বড় value বের করতে:

```
Math.max(10, 20, 5, 30);

// 30
```

আর:

```
Math.max(100, 50, 75);

// 100
```

14. `Math.min()`

সবচেয়ে ছোট value:

```
Math.min(10, 20, 5, 30);  
// 5
```

15. Array-এর সাথে `Math.max()`

এখানে একটি important বিষয়:

```
const numbers = [10, 20, 5, 30];  
  
Math.max(numbers);
```

এটা কাজ করবে না expected way-তে।

কারণ `Math.max()` individual arguments নেয়।

সঠিক:

```
Math.max(...numbers);  
// 30
```

Spread operator array-এর values-কে individual arguments বানিয়ে দেয়।

16. Array-এর সাথে `Math.min()`

```
const numbers = [10, 20, 5, 30];  
  
const minimum = Math.min(...numbers);  
  
console.log(minimum);  
// 5
```

17. `Math.max()` এবং `Math.min()` দিয়ে Range

```
const values = [10, 50, 20, 90, 30];

const max = Math.max(...values);
const min = Math.min(...values);

console.log(min);
console.log(max);
```

এভাবে array-এর minimum এবং maximum বের করা যায়।

18. `Math.random()`

`Math.random()` 0 থেকে 1-এর মধ্যে একটি pseudo-random number দেয়।

```
console.log(Math.random());
```

Possible:

```
0.123...
0.845...
0.567...
```

Important:

```
0 <= Math.random() < 1
```

অর্থাৎ 1 return করে না।

19. Random Number Between 0 and 1

```
const random = Math.random();

console.log(random);
```

Range:

```
0 ≤ random < 1
```

20. Random Number Between 0 and 10

```
const random = Math.random() * 10;

console.log(random);
```

Range:

```
0 ≤ random < 10
```

কিন্তু decimal থাকবে।

21. Random Integer Between 0 and 9

```
const random = Math.floor(Math.random() * 10);

console.log(random);
```

Possible:

```
0
1
2
3
4
5
6
7
8
9
```

22. Random Integer Between 1 and 10

সবচেয়ে common pattern:

```
const random = Math.floor(Math.random() * 10) + 1;

console.log(random);
```

Possible:

1 - 10

Formula

```
Math.floor(Math.random() * max) + 1
```

23. Random Integer Between **min** and **max**

Reusable formula:

```
Math.floor(Math.random() * (max - min + 1)) + min
```

Example:

```
const min = 5;
const max = 15;

const random =
  Math.floor(Math.random() * (max - min + 1)) + min;

console.log(random);
```

Possible:

5 - 15

দুটো boundary-ই included।

24. Reusable Random Integer Function

Production code-এ helper function রাখা useful:

```
function randomInt(min, max) {  
  return Math.floor(  
    Math.random() * (max - min + 1)  
  ) + min;  
}  
  
console.log(randomInt(1, 10));  
console.log(randomInt(100, 200));
```

25. Random Decimal Between Min and Max

Integer না, decimal দরকার হলে:

```
function randomFloat(min, max) {  
  return Math.random() * (max - min) + min;  
}  
  
console.log(randomFloat(10, 20));
```

Range:

```
10 ≤ result < 20
```

26. Random Color Example

Random RGB value:

```
function randomRGB() {  
    const r = Math.floor(Math.random() * 256);  
    const g = Math.floor(Math.random() * 256);  
    const b = Math.floor(Math.random() * 256);  
  
    return `rgb(${r}, ${g}, ${b})`;  
}  
  
console.log(randomRGB());
```

Possible:

```
rgb(120, 45, 200)
```

27. Random Array Item

`Math.random()` দিয়ে array থেকে random item নেওয়া যায়।

```
const fruits = ["Apple", "Banana", "Mango", "Orange"];  
  
const index = Math.floor(Math.random() * fruits.length);  
  
console.log(fruits[index]);
```

এটা frontend/game development-এ খুব common pattern।

28. Random Dice Roll

একটি dice-এ `1-6`:

```
function rollDice() {  
    return Math.floor(Math.random() * 6) + 1;  
}  
  
console.log(rollDice());
```

Possible:

- 1
- 2
- 3
- 4
- 5
- 6

29. Random OTP-এর ক্ষেত্রে সতর্কতা

Simple UI demo বা game-এর জন্য:

```
Math.floor(Math.random() * 1000000);
```

ব্যবহার করা যায়।

কিন্তু **security-sensitive OTP, password, token, session ID** generate করার জন্য `Math.random()` ব্যবহার করা উচিত নয়।

Security-sensitive random value-এর জন্য browser-এর Web Crypto API ব্যবহার করা উচিত।

30. `Math` Constants

কিছু useful constants:

```
Math.PI  
Math.E  
Math.SQRT2  
Math.SQRT1_2  
Math.LN2  
Math.LN10  
Math.LOG2E  
Math.LOG10E
```

সবচেয়ে বেশি ব্যবহৃত:

```
Math.PI  
Math.E
```

31. `Math.max()` with Negative Numbers

```
Math.max(-10, -20, -5);  
// -5
```

কারণ `-5` সবচেয়ে বড়।

Similarly:

```
Math.min(-10, -20, -5);  
// -20
```

32. `Math.max()` Without Arguments

```
Math.max();  
// -Infinity
```

এবং:

```
Math.min();  
// Infinity
```

এগুলো mathematical identity-এর কারণে এমন result দেয়।

Practical code-এ সাধারণত এই behavior manually rely করার দরকার হয় না।

33. `Math.max()` with `NaN`

```
Math.max(10, 20, NaN);  
// NaN
```

একটি `NaN` পুরো result-কে `NaN` করে দিতে পারে।

তাই external/user/API data নিয়ে কাজ করার আগে validation useful।

34. `Math.floor()` দিয়ে Price Handling

ধরো:

```
const price = 99.99;
```

যদি শুধু integer অংশ দরকার:

```
const wholePrice = Math.floor(price);  
  
console.log(wholePrice);  
// 99
```

কিন্তু money rounding-এর জন্য blindly `floor()` ব্যবহার করা ঠিক নয়।

যদি nearest value দরকার:

```
Math.round(price);
```

35. `Math.round()` vs `toFixed()`

দুটো দেখতে similar মনে হলেও purpose আলাদা।

```
const price = 99.456;
```

`Math.round()`

```
Math.round(price);  
// 99
```

এটা integer দেয়।

`toFixed()`

```
price.toFixed(2);  
// "99.46"
```

এটা formatted string দেয়।

36. Rounding to 2 Decimal Places

একটি number-কে 2 decimal পর্যন্ত round করার common technique:

```
const number = 10.456;

const rounded = Math.round(number * 100) / 100;

console.log(rounded);
// 10.46
```

তবে money display-এর জন্য:

```
number.toFixed(2);
```

অনেক সময় বেশি convenient।

37. Temperature Conversion Example

Celsius থেকে Fahrenheit:

```
function celsiusToFahrenheit(celsius) {
  return (celsius * 9) / 5 + 32;
}

console.log(celsiusToFahrenheit(30));
// 86
```

38. Distance Calculation

দুই point-এর difference:

```
const x1 = 10;
    const x2 = 25;

    const distance = Math.abs(x2 - x1);

    console.log(distance);
    // 15
```

39. Power Calculation

```
const base = 5;
    const exponent = 2;

    const result = base ** exponent;

    console.log(result);
    // 25
```

Equivalent:

```
Math.pow(5, 2);
    // 25
```

Modern JavaScript-এ:

```
5 ** 2
```

সাধারণত cleaner।

40. Square Root Example

```
const number = 144;

const result = Math.sqrt(number);

console.log(result);

// 12
```

41. `Math.cbrt()`

Cube root-এর জন্য:

```
Math.cbrt(27);

// 3
```

এটাও useful mathematical method।

42. `Math.random()` — Important Limitation

`Math.random()` cryptographically secure random generator নয়।

অর্থাৎ:

```
Math.random()
```

এর randomness security-sensitive purpose-এর জন্য যথেষ্ট নয়।

Game:

```
Math.random()
```

ঠিক আছে।

Security token:

```
Math.random()
```

✗ Avoid.

43. Common Mistake — `Math.random() * 10`

```
Math.random() * 10
```

এটা integer return করে না।

Possible:

```
2.348  
8.921  
0.145
```

Integer চাইলে:

```
Math.floor(Math.random() * 10);
```

44. Common Mistake — `Math.floor()` Negative Numbers

অনেকে ভাবে:

```
Math.floor(-2.5);
```

মানে `-2`।

কিন্তু:

```
Math.floor(-2.5);  
// -3
```

কারণ floor $\rightarrow$ negative infinity-এর দিকে।

45. Common Mistake —

Math.max(array)

ভুল:

```
const numbers = [10, 20, 30];

Math.max(numbers);
```

সঠিক:

```
Math.max(...numbers);
```

46. Quick Reference Table

Method	কাজ
Math.floor()	নিচের integer
Math.ceil()	উপরের integer
Math.round()	nearest integer
Math.trunc()	decimal বাদ
Math.abs()	absolute value
Math.sign()	sign detect
Math.sqrt()	square root
Math.cbrt()	cube root
Math.pow()	power
Math.max()	maximum
Math.min()	minimum
Math.random()	random decimal
Math.PI	π
Math.E	Euler's number

47. Real-World Example — Product Rating

ধরো rating এসেছে:

```
const rating = 4.6789;
```

Display:

```
const displayRating = rating.toFixed(1);

console.log(displayRating);
// "4.7"
```

আর যদি integer rating দরকার:

```
Math.round(rating);
// 5
```

48. Real-World Example — Pagination

ধরো:

```
const totalItems = 53;
const itemsPerPage = 10;
```

কতগুলো page লাগবে?

```
const totalPages = Math.ceil(totalItems / itemsPerPage);

console.log(totalPages);
// 6
```

এখানে `ceil()` খুব useful।

কারণ:

```
53 / 10 = 5.3
```

5.3 page হতে পারে না, তাই 6 pages দরকার।

49. Real-World Example — Progress Percentage

```
const completed = 7;
const total = 10;

const percentage = (completed / total) * 100;

console.log(percentage);
// 70
```

Round করতে:

```
const percentage = Math.round(
  (completed / total) * 100
);
```

50. Real-World Example — Clamp Value

কোনো value-কে minimum এবং maximum-এর মধ্যে রাখতে `Math.min()` + `Math.max()` খুব useful।

```
function clamp(value, min, max) {
  return Math.min(Math.max(value, min), max);
}
```

Example:

```
console.log(clamp(50, 0, 100));
// 50

console.log(clamp(-10, 0, 100));
// 0

console.log(clamp(150, 0, 100));
// 100
```

এটা UI, volume, progress bar, game এবং animation-এ খুব useful।

51. Practice — Predict the Output

Question 1

```
console.log(Math.floor(10.9));
```

Question 2

```
console.log(Math.ceil(10.1));
```

Question 3

```
console.log(Math.round(10.5));
```

Question 4

```
console.log(Math.floor(-10.1));
```

Question 5

```
console.log(Math.trunc(-10.9));
```

Question 6

```
console.log(Math.abs(-50));
```

Question 7

```
console.log(Math.max(10, 50, 20));
```

Question 8

```
console.log(Math.min(10, 50, 20));
```

Question 9

```
console.log(Math.sqrt(81));
```

Question 10

```
console.log(2 ** 5);
```

52. Coding Challenges

Challenge 1 — Random Integer

একটি function লিখো যেটা `1-100` এর মধ্যে random integer return করবে।

Expected:

```
randomInt(1, 100);
```

Challenge 2 — Dice

```
rollDice();
```

এটা `1-6` এর মধ্যে random number return করবে।

Challenge 3 — Maximum

```
findMax([10, 50, 20, 90, 30]);
```

Expected:

```
90
```

Challenge 4 — Pagination

Function:

```
getTotalPages(53, 10);
```

Expected:

```
6
```

Challenge 5 — Clamp

```
clamp(150, 0, 100);
```

Expected:

```
100
```

53. Interview Questions

Beginner

1. `Math` object কী?
2. `Math.floor()` কী করে?
3. `Math.ceil()` কী করে?
4. `Math.round()` কী করে?
5. `Math.random()` কী range-এর value দেয়?
6. `Math.max()` এবং `Math.min()` কী করে?
7. `Math.abs()` কী?
8. `Math.PI` কী?

Intermediate

9. `Math.floor(-2.5)` কেন `-3`?
10. `Math.floor()` এবং `Math.trunc()` -এর পার্থক্য কী?
11. Array-এর maximum বের করতে `Math.max(...array)` কেন ব্যবহার করতে হয়?

12. `Math.random()` দিয়ে `1-10` random integer কীভাবে generate করবে?
13. `Math.random()` security-sensitive data-এর জন্য কেন ব্যবহার করা উচিত নয়?
14. `Math.ceil()` pagination-এর ক্ষেত্রে কেন useful?
15. `Math.round()` এবং `toFixed()` -এর পার্থক্য কী?
16. `Math.min()` এবং `Math.max()` দিয়ে value clamp কীভাবে করবে?

54. Mental Model

Math methods মনে রাখার সবচেয়ে সহজ উপায়:

```
floor → নিচে  
ceil → উপরে  
round → কাছাকাছি  
trunc → decimal কেটে ফেলো  
  
abs → negative বাদ  
sign → কোন direction/sign  
  
min → সবচেয়ে ছোট  
max → সবচেয়ে বড়  
  
sqrt → square root  
cbrt → cube root  
pow → power  
  
random → random value  
PI →  $\pi$ 
```

55. Golden Rules

1. `Math.floor()` → নিচের *integer*-এর দিকে যায়
2. `Math.ceil()` → উপরের *integer*-এর দিকে যায়
3. `Math.round()` → *nearest integer*
4. `Math.trunc()` → *decimal* অংশ বাদ দেয়

5. Negative numbers-এর ক্ষেত্রে `floor()` এবং `trunc()` এক নয়

6. `Math.random()` → `0 <= value < 1`

7. Random integer-এর common pattern: `Math.floor(Math.random() * range) + min`

8. Array-এর max/min: `Math.max(...array)` / `Math.min(...array)`

9. Security-sensitive randomness-এর জন্য `Math.random()` নয়

10. Pagination-এর মতো ক্ষেত্রে `Math.ceil()` খুব useful

11. `Math.min()` + `Math.max()` দিয়ে সহজে value clamp করা যায়

12. Modern JavaScript-এ power-এর জন্য `**` operator ব্যবহার করা যায়